

Modulos componentes y desarrollo (220501096-AA1-EV01 y EV02)

Robert Andrei Vergara Garavito

Duván Guerra

Alejandro Ariza Mojica

SENA CENTRO DE ELECTRICIDAD, ELECTRÓNICA Y TELECOMUNICACIONES

3341112

Oscar Julián Piraban Rodríguez

Septiembre 11, 2026

TABLA DE CONTENIDO

CONTENIDO	PÁGINA
Introducción	3
Descripción General del Proyecto	3-5
Requerimientos del Sistema	5-7
Casos de Uso e Historias de Usuario	7-9
Arquitectura del Sistema	9-13
Tecnologías, Frameworks y Herramientas	13-15
Desarrollo de Módulos	16-21
Integración de Módulos	21
Mecanismos de Seguridad	21-22
Control de Versiones	22
Entorno de Desarrollo y Pruebas	23
Pruebas y Resultados	24
Configuracion de Base de Datos y Servidor	25
Manual Técnico	25-26
Estado Actual del Proyecto	26-27
Conclusiones	27

1. Introducción

El presente documento describe el desarrollo, integración y documentación de los módulos de El Granero Digital, proyecto orientado a conectar productores campesinos y compradores urbanos mediante una plataforma digital.

La solución utiliza una arquitectura basada en frontend React, backend Django y una API REST, con funcionalidades relacionadas con usuarios, productos, pedidos y futuras notificaciones e integraciones externas.

Durante el desarrollo se han utilizado herramientas como Python, Django REST Framework, SQLite, Git, GitHub y Postman, aplicando principios de modularidad, separación de responsabilidades, control de versiones y pruebas.

La documentación presenta los requerimientos, arquitectura, tecnologías, seguridad, integración, pruebas y estado actual del proyecto, diferenciando las funcionalidades implementadas de aquellas que continúan en desarrollo o están proyectadas.

2. Descripción General del Proyecto

2.1. Nombre del proyecto

El Granero Digital

Slogan: *"Un país que crece desde el campo."*

2.2. Descripción y problemática

El Granero Digital es una solución orientada al comercio agrícola que busca conectar directamente a productores campesinos y compradores urbanos, facilitando la publicación, consulta y gestión de productos mediante una plataforma digital.

El proyecto surge como respuesta a las dificultades de comercialización directa y acceso a canales digitales que pueden enfrentar algunos productores.

2.3. Objetivo general

Desarrollar una plataforma digital que facilite la comercialización de productos agrícolas mediante la conexión directa entre campesinos y compradores.

2.4. Usuarios del sistema

- Campesino: publica y administra sus productos y gestiona solicitudes de compra.
- Comprador: consulta productos, utiliza filtros y realiza solicitudes o compras.
- Administrador: supervisa usuarios, pedidos y actividades del sistema.

2.5. Módulos principales

- Usuarios y autenticación: registro, acceso y roles.
- Productos: creación, consulta, edición y eliminación.
- Órdenes: gestión de pedidos.
- Notificaciones: comunicación de eventos relevantes.
- Frontend: interfaz de usuario.
- Administración: supervisión y gestión del sistema.

2.6. Alcance tecnológico

El sistema utiliza React para el frontend y Django + Django REST Framework para el backend, comunicados mediante una API REST con intercambio de datos en formato JSON.

Actualmente se utiliza SQLite en desarrollo y se contempla MySQL para la implementación posterior. También se proyectan integraciones con WhatsApp, medios de pago, notificaciones y operación sin conexión.

3. Requerimientos del Sistema

Los requerimientos definen las funcionalidades y condiciones que debe cumplir **El Granero Digital** y sirven como base para el diseño, desarrollo, integración y pruebas del sistema.

3.1. Requerimientos funcionales

Código	Requerimiento	Módulo
RF01	Registrar productos con nombre, precio, fotografía y cantidad.	Productos
RF02	Editar productos publicados.	Productos
RF03	Eliminar productos publicados.	Productos
RF04	Sincronizar productos al recuperar la conexión.	Productos / Offline
RF05	Consultar el catálogo de productos agrícolas.	Productos / Compradores
RF06	Filtrar por categoría, precio y disponibilidad.	Productos
RF07	Contactar al productor mediante WhatsApp.	Compradores / Integraciones

RF08	Registrar usuarios como Campesino o Comprador.	Usuarios
RF09	Iniciar sesión mediante credenciales.	Autenticación
RF10	Enviar pedidos al productor.	Órdenes
RF11	Consultar historial de pedidos.	Órdenes
RF12	Gestionar usuarios y pedidos desde el rol administrador.	Administración
RF13	Consultar reportes de actividad.	Administración
RF14	Notificar al campesino sobre eventos relacionados con pedidos.	Notificaciones
RF15	Proporcionar una interfaz sencilla y accesible.	Frontend

3.2. Requerimientos no funcionales

Código	**Requerimiento**	**Descripción**
RNF01	Usabilidad	Interfaz sencilla para los diferentes usuarios.
RNF02	Rendimiento	Catálogo con respuesta inferior a 10 segundos bajo conexión estable.
RNF03	Seguridad	Protección de credenciales mediante mecanismos seguros.

RNF04	Compatibilidad	Consideración de dispositivos Android de recursos limitados.

3.3. Estado de implementación

El proyecto se encuentra en desarrollo, por lo que los requerimientos presentan diferentes niveles de avance.

Implementado y probado: gestión de usuarios y roles, productos, operaciones CRUD, API REST y pruebas de endpoints mediante Postman.

En desarrollo o pendiente: órdenes, notificaciones, sincronización offline, integraciones externas y algunas funcionalidades administrativas.

Las funcionalidades proyectadas se diferencian de las implementadas para reflejar correctamente el estado actual del sistema.

4. Casos de Uso e Historias de Usuario

Los casos de uso e historias de usuario permiten representar las principales interacciones entre los usuarios y las funcionalidades de **El Granero Digital**.

4.1. Caso de uso principal

CU-01 – Registrar producto

Actor: Campesino.

Objetivo: Registrar un producto agrícola para ofrecerlo a los compradores.

Precondición: El usuario debe estar registrado y tener rol de Campesino.

Flujo:

1. El campesino inicia sesión.
2. Accede a la gestión de productos.
3. Selecciona registrar producto.
4. Ingresa los datos solicitados.
5. El sistema valida la información.
6. Se almacena el producto.
7. El sistema informa el resultado.

Resultado: Producto registrado y disponible para consulta según las condiciones de publicación.

La sincronización sin conexión corresponde a una funcionalidad proyectada.

4.2. Historias de usuario principales

Código	Historia	Criterios principales
HU-01	**Publicar producto:** Como campesino, quiero registrar productos para ofrecerlos a compradores.	Validar datos, asociar producto al usuario y permitir su consulta.
HU-08	**Registro por rol:** Como usuario, quiero registrarme como Campesino o Comprador para acceder a las funciones correspondientes.	Validar datos, asignar rol y evitar registro público como administrador.

HU-09	**Inicio de sesión:** Como usuario registrado, quiero autenticarme para acceder de forma segura.	Validar credenciales, proteger contraseñas y controlar acceso según rol.
HU-10	**Envío de pedidos:** Como comprador, quiero enviar pedidos a un campesino.	Seleccionar productos, registrar pedido y relacionar comprador y productor.
HU-11	**Historial:** Como usuario, quiero consultar mis pedidos anteriores.	Mostrar pedidos asociados y su estado.
HU-12	**Gestión administrativa:** Como administrador, quiero gestionar usuarios y consultar actividad.	Restringir acceso administrativo y permitir gestión y consulta de información.

Estas historias sirven como referencia para el diseño, desarrollo y posterior validación de las funcionalidades mediante pruebas.

5. Arquitectura del Sistema

El Granero Digital utiliza una arquitectura **cliente-servidor**, separando la interfaz de usuario de la lógica del sistema y el acceso a datos. La comunicación entre frontend y backend se realiza mediante una **API REST** utilizando HTTP y JSON.

Esta estructura permite desarrollar los módulos de forma independiente y facilita su posterior integración con una aplicación móvil.

5.1. Arquitectura por capas

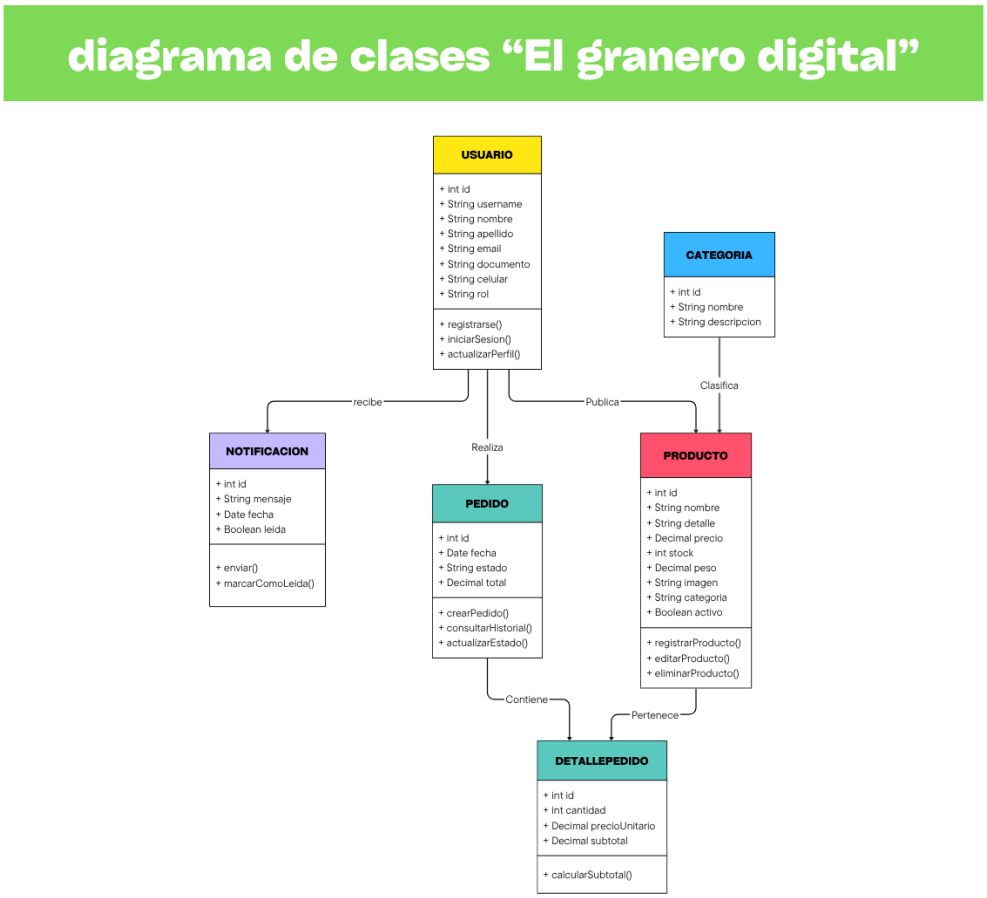
El sistema se organiza en cuatro capas:

- **Presentación:** interfaz desarrollada con React.
- **Lógica de negocio:** backend desarrollado con Django y Django REST Framework.
- **Acceso a datos:** Django ORM.
- **Datos:** SQLite en el entorno actual y MySQL como opción contemplada para implementación.

Flujo: Usuario → React → API REST → Django/ORM → Base de datos.

5.2. Diagrama de clases

El diagrama representa las principales entidades del dominio, incluyendo **Usuario, Producto, Pedido, DetallePedido, Notificación y Categoría**, y sus relaciones.



El diagrama representa el modelo conceptual proyectado; no todas sus entidades se encuentran implementadas actualmente.

5.3. Diagrama de componentes

Representa los principales componentes tecnológicos y su comunicación:

Frontend React.

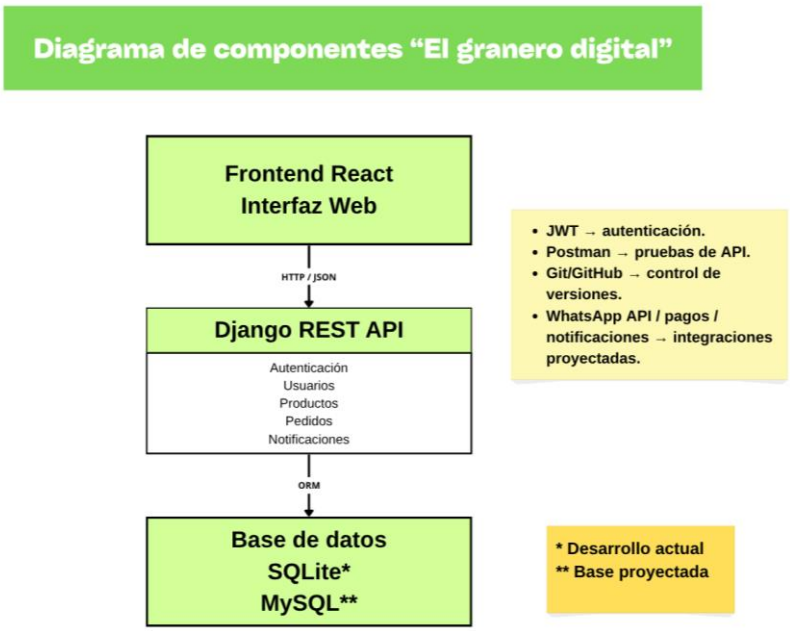
API REST Django.

Módulos de usuarios, productos, órdenes y notificaciones.

Base de datos.

Herramientas de prueba y control de versiones.

También se contemplan integraciones futuras con WhatsApp, medios de pago y servicios de notificaciones.

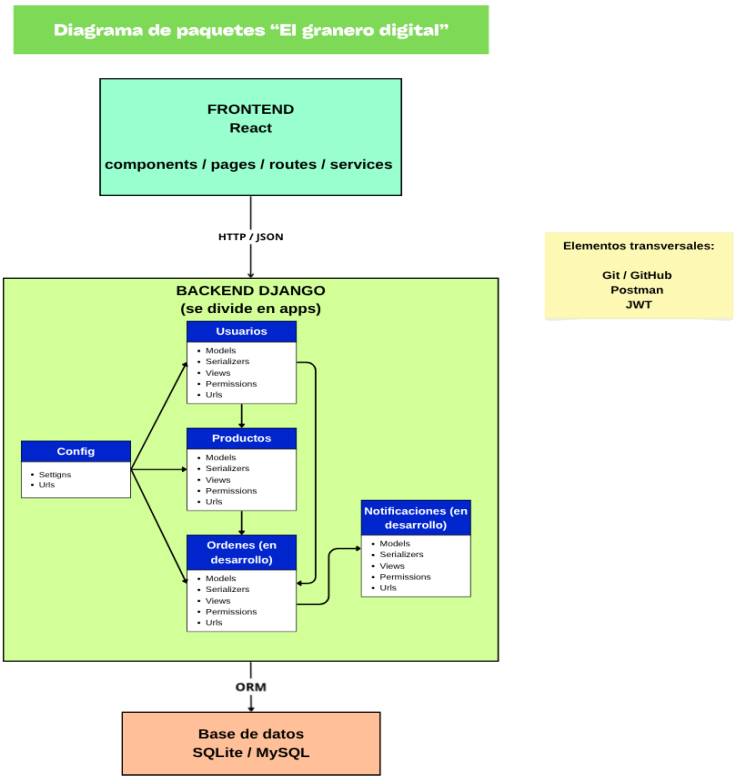


5.4. Diagrama de paquetes

Representa la organización modular del código backend:

- **usuarios**
- **productos**
- **ordenes**
- **notificaciones**
- **config**

El frontend se mantiene separado y consume los servicios mediante la API REST.



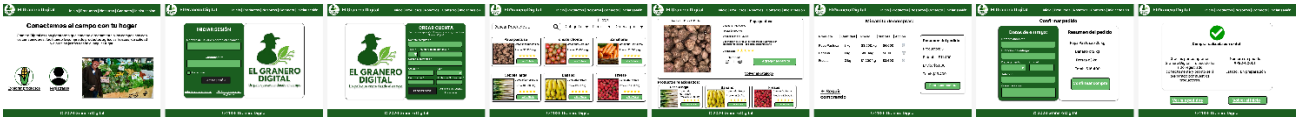
Esta organización permite que cada módulo evolucione de manera independiente, manteniendo una estructura que facilita posteriormente la integración de nuevas funcionalidades.

5.5. Mapa de navegación

El mapa representa las principales rutas de interacción:

Inicio → Registro / Inicio de sesión → Productos → Compra → Confirmación

El acceso a determinadas funcionalidades depende del rol y estado de autenticación.



5.6. Relación entre los componentes

El frontend consume los endpoints de Django REST Framework, mientras que el backend procesa las solicitudes y utiliza Django ORM para interactuar con la base de datos.

Esta separación permite reutilizar la API para futuros clientes, como la aplicación móvil proyectada.

6. Tecnologías, Frameworks y Herramientas

Para el desarrollo de **El Granero Digital** se utilizaron tecnologías y herramientas orientadas a la construcción de una solución modular, facilitando la separación entre frontend, backend, datos, pruebas y control de versiones.

6.1. Backend

El backend está desarrollado con **Python**, utilizando **Django 6.0.5** como framework principal y **Django REST Framework 3.17.1** para la construcción de la API REST.

Django proporciona la estructura de la aplicación, modelos, rutas, autenticación y acceso a datos mediante ORM. Django REST Framework permite crear endpoints, serializar y validar información, gestionar solicitudes HTTP y establecer mecanismos de permisos y autenticación.

6.2. Frontend

El frontend se desarrolla con **React 19.2.8**, utilizando **React Router DOM 7.18.3** para la navegación entre vistas y **Vite 8.2.2** como herramienta de desarrollo y construcción.

La utilización de componentes permite organizar la interfaz de forma modular y facilita su posterior integración con la API REST del backend.

6.3. Base de datos

Actualmente se utiliza **SQLite** para el entorno de desarrollo y pruebas debido a su configuración sencilla.

La arquitectura del proyecto contempla **MySQL** como sistema gestor para una implementación posterior.

6.4. Pruebas y control de versiones

Postman se utiliza para verificar los endpoints de la API mediante solicitudes **GET, POST, PATCH y DELETE**, permitiendo validar el comportamiento de los servicios desarrollados.

Git se utiliza para el control de versiones y seguimiento de cambios, mientras que **GitHub** permite alojar el repositorio y facilitar la colaboración entre los integrantes del proyecto.

6.5. Dependencias principales

Tecnología / herramienta	Versión

Python	Lenguaje utilizado
Django	6.0.5
Django REST Framework	3.17.1
React	19.2.8
React DOM	19.2.8
React Router DOM	7.18.3
Vite	8.2.2
ESLint	10.9.0
SQLite	Desarrollo
Git / GitHub	Control de versiones
Postman	Pruebas de API

6.6. Justificación tecnológica

La combinación de **React + Django REST Framework** permite mantener separadas la interfaz y la lógica del servidor mediante una API REST. Esto facilita la integración de los módulos y permite que la misma API pueda ser utilizada posteriormente por otros clientes, como la aplicación móvil proyectada.

Git/GitHub facilita el control y colaboración sobre el código, mientras que Postman permite verificar los servicios durante el desarrollo y las pruebas de integración.

7. Desarrollo de Módulos

El Granero Digital se organiza mediante módulos que separan las responsabilidades principales del sistema y facilitan su desarrollo, mantenimiento, integración y pruebas.

Los módulos contemplados son: **usuarios y autenticación, productos, órdenes, notificaciones y frontend**.

7.1. Módulo de usuarios y autenticación

Este módulo administra los usuarios y sus roles dentro de la plataforma. Se utiliza un modelo de usuario personalizado basado en Django.

Roles: Campesino, Comprador y Administrador.

Entre sus principales funciones se encuentran:

- Registro de usuarios.
- Administración de roles.
- Gestión de información personal.
- Autenticación.
- Control de permisos.

El modelo **Usuario** incluye identificador, nombre de usuario, nombres, apellidos, correo, documento, celular y rol. Los campos documento y celular se manejan como valores únicos.

El registro público permite seleccionar únicamente **Campesino** o **Comprador**, evitando que un usuario pueda asignarse directamente el rol administrativo.

La API utiliza serializadores para validar y transformar los datos, junto con mecanismos de permisos para controlar el acceso a los recursos.

Entradas: datos personales, credenciales y tipo de registro.

Procesamiento: validación de datos, creación del usuario y asignación del rol.

Salidas: información serializada y respuestas HTTP.

Estado: Implementado y en proceso de fortalecimiento. El modelo personalizado, registro, roles, API y permisos iniciales han sido desarrollados y probados mediante Postman.

7.2. Módulo de productos

Permite administrar los productos agrícolas ofrecidos en la plataforma.

Sus principales operaciones son:

- Crear productos.
- Consultar productos.
- Editar productos.
- Eliminar productos.
- Gestionar información y disponibilidad.
- Exponer operaciones mediante API REST.

Los productos contemplan información como nombre, detalle, precio, stock, peso, imagen, categoría y estado.

El módulo utiliza **Django REST Framework y ModelViewSet**, permitiendo operaciones mediante los métodos **GET, POST, PATCH, PUT y DELETE**.

Entradas: información y características del producto.

Procesamiento: validación, serialización y almacenamiento mediante Django ORM.

Salidas: datos en formato JSON y códigos de respuesta HTTP.

Pruebas: se realizaron pruebas mediante Postman sobre consulta, creación, modificación y eliminación de productos.

Estado: Implementado y probado. El CRUD y la API REST se encuentran funcionales y preparados para su integración con el frontend React.

7.3. Módulo de órdenes

Este módulo está destinado a gestionar las solicitudes de compra y relacionar compradores, productores y productos.

Contempla conceptualmente:

- Creación de pedidos.
- Relación entre comprador y productor.
- Productos y cantidades solicitadas.
- Estado y fecha del pedido.
- Historial de pedidos.
- Gestión administrativa.

Entradas: comprador, productos, cantidades y datos necesarios para generar el pedido.

Procesamiento: validación y registro de las relaciones correspondientes.

Salidas: información y estado del pedido.

Estado: En desarrollo. La estructura se encuentra contemplada en los requerimientos y arquitectura, pero sus funcionalidades completas aún están pendientes de integración.

7.4. Módulo de notificaciones

Su propósito es informar a los usuarios sobre eventos relevantes, especialmente aquellos relacionados con solicitudes o pedidos.

Contempla:

- Generación de notificaciones.
- Usuario receptor.
- Mensaje y fecha.
- Estado de lectura.
- Consulta de notificaciones.

Entradas: evento, usuario destinatario y mensaje.

Procesamiento: identificación del evento y generación de la notificación.

Salidas: registro y estado de la notificación.

Estado: En desarrollo. Su integración con órdenes y otros servicios se encuentra pendiente.

7.5. Frontend

El frontend constituye la capa de presentación y se desarrolla con **React, React Router DOM y Vite**.

Sus responsabilidades principales son:

- Presentar la interfaz.
- Gestionar la navegación.
- Capturar información del usuario.
- Consumir la API REST.
- Mostrar las respuestas del backend.
- Organizar la interfaz mediante componentes reutilizables.

El código se encuentra en el directorio:

frontend/

La comunicación con el backend sigue el flujo:

Usuario → React → Solicitud HTTP → API Django → Respuesta JSON → React → Usuario

Estado: En desarrollo e integración. El frontend forma parte de la estructura actual del proyecto y continúa su construcción y conexión con los servicios del backend.

7.6. Integración progresiva

El desarrollo se realiza de manera progresiva, validando los componentes antes de completar su integración.

Actualmente los mayores avances se concentran en **usuarios y productos**, incluyendo API REST y pruebas con Postman. Los módulos de órdenes y notificaciones, así como determinadas funcionalidades de autenticación, frontend e integraciones externas, continúan en desarrollo.

Esta estrategia permite identificar errores de manera temprana y facilita la integración posterior de los diferentes componentes del sistema.

8. Integración de Módulos

La integración de El Granero Digital se basa en la comunicación entre los módulos del backend y el frontend mediante una API REST.

El flujo general es:

React → API REST Django → Django/ORM → Base de datos

Los módulos de usuarios, productos, órdenes y notificaciones se encuentran organizados de manera independiente, permitiendo su integración progresiva.

Actualmente, los módulos con mayor nivel de integración son usuarios y productos, incluyendo endpoints y pruebas mediante Postman. Los módulos de órdenes y notificaciones continúan en desarrollo.

La separación entre frontend y backend permite que los servicios puedan ser reutilizados posteriormente por otros clientes, como la aplicación móvil proyectada.

9. Mecanismos de Seguridad

El sistema incorpora mecanismos básicos de seguridad proporcionados por Django y Django REST Framework.

Entre los principales se encuentran:

Autenticación: control de acceso mediante usuarios autenticados.

Roles: diferenciación entre Campesino, Comprador y Administrador.

Permisos: restricción de operaciones según usuario y rol.

Contraseñas: gestión mediante los mecanismos de seguridad de Django.

Validación: serializadores para validar información recibida por la API.

Protección de privilegios: el registro público no permite asignarse el rol Administrador.

Control de acceso a recursos: los endpoints protegidos requieren autenticación según su configuración.

Estos mecanismos constituyen la base de seguridad actual y deberán fortalecerse durante las siguientes etapas de desarrollo y despliegue.

10. Control de Versiones

El proyecto utiliza Git para el control de versiones y GitHub como repositorio remoto.

El código se organiza en ramas para facilitar el desarrollo y la integración de cambios. La estructura principal separa backend, frontend, documentación y recursos del proyecto.

El control de versiones permite:

Registrar cambios.

Mantener historial del desarrollo.

Recuperar versiones anteriores.

Facilitar el trabajo colaborativo.

Integrar los avances de los diferentes integrantes.

Repositorio: GitHub – El Granero Digital. Link: **https://github.com/AlejandroArizaPy/el-granero-digital**

11. Entorno de Desarrollo y Pruebas

El proyecto se desarrolla utilizando un entorno separado para backend y frontend.

Backend: Python, Django, Django REST Framework y entorno virtual de Python.

Frontend: React, React Router DOM y Vite.

Base de datos: SQLite para desarrollo.

Pruebas: Postman para validar los endpoints de la API.

La estructura principal del repositorio es:

el-granero-digital/

├— backend/

├— frontend/

├— docs/

├— mockups/

├— .gitignore

└— README

Las pruebas se realizan sobre el entorno local antes de integrar nuevas funcionalidades.

12. Pruebas y Resultados

Las pruebas iniciales se realizaron principalmente sobre la API REST mediante Postman.

Módulo	Prueba	Resultado
Productos	GET	Correcto
Productos	POST	Correcto
Productos	PATCH	Correcto
Productos	DELETE	Correcto
Usuarios	Registro/API	Correcto
Usuarios	Permisos por rol	Validado
Autenticación	Acceso a recursos protegidos	En fortalecimiento

Las pruebas permitieron identificar y corregir errores relacionados con validación, permisos, rutas y datos enviados a la API.

Las evidencias gráficas de las pruebas realizadas se presentan en los apartados correspondientes o como anexos del documento.

13. Configuración de Base de Datos y Servidor

Durante el desarrollo se utiliza SQLite como base de datos local. Django administra la comunicación con la base de datos mediante su ORM, evitando realizar directamente las operaciones SQL desde los módulos.

La configuración permite ejecutar el backend mediante el servidor de desarrollo de Django.

La arquitectura contempla posteriormente la migración hacia MySQL para un entorno de implementación.

Flujo de datos:

API → Django → ORM → SQLite

La configuración de producción, dominio y servidor definitivo queda pendiente de la etapa de despliegue.

14. Manual Técnico

Para ejecutar el proyecto en un entorno de desarrollo se deben realizar, de forma general, los siguientes pasos:

- Descargar o clonar el repositorio.
- Ubicar el proyecto en el entorno de desarrollo.
- Crear y activar el entorno virtual de Python.
- Instalar las dependencias del backend.
- Configurar la base de datos.
- Ejecutar las migraciones de Django.

- Iniciar el servidor backend.
- Ejecutar el frontend mediante Vite.
- Realizar las pruebas de comunicación con la API.

La estructura separada entre backend/ y frontend/ permite ejecutar y mantener cada componente de forma independiente.

Las instrucciones específicas podrán ampliarse conforme avance la configuración del entorno de producción.

15. Estado Actual del Proyecto

El proyecto se encuentra en fase de desarrollo e integración progresiva.

Componente	Estado
Modelo de usuarios y roles	Implementado
API de usuarios	Implementada
CRUD de productos	Implementado y probado
API de productos	Implementada y probada
Autenticación y permisos	Implementados, en fortalecimiento
Frontend React	En desarrollo
Órdenes	En desarrollo

Notificaciones En desarrollo

Integraciones externas Proyectadas

SQLite Implementada para desarrollo

MySQL Proyectada

Aplicación móvil Proyectada

El avance actual permite continuar con la integración de los módulos restantes y las pruebas correspondientes.

16. Conclusiones

El desarrollo de El Granero Digital ha permitido establecer una arquitectura modular basada en React, Django y Django REST Framework, facilitando la separación entre presentación, lógica y acceso a datos.

Los módulos de usuarios y productos presentan avances funcionales y han sido sometidos a pruebas mediante Postman. Los demás componentes continúan su proceso de desarrollo e integración.

El uso de Git, GitHub y herramientas de pruebas permite mantener control sobre el código y validar progresivamente los servicios desarrollados.

La arquitectura establecida proporciona una base para continuar con la implementación de órdenes, notificaciones, integraciones externas, aplicación móvil y despliegue del sistema.